Week 5: memory, program structure, errors, exceptions and assertions

These notes cover week 5's lecture, for revision. Read them beside its slides: every part names the pages it covers. They add the explanation behind each slide, the runs written out line by line, the mistakes the code invites with the message Python prints, the polls with their answers, and a few questions to check yourself, answered at the end. Common questions are answered where they come up. Others have written answers on the Q&A page, and the Function reference lists what each tool takes and gives back; both are on Brightspace, under Feedback. If you already program, the boxes headed **Good practice** are for you: the habits experienced programmers keep, next to the code they are about. The notes replace neither the lecture nor the practical: the sheet is where you practice.

Where we started

Slides, pages 1, 3, 5 and 6.

Poll. One word: when two names point at the *same* list, what is that called? The word is **aliasing**, and the whole of section 1 is about it: two names for one object, so that a change made through one name shows through the other.

Last week's café does real work: a dictionary menu, orders kept as lists of tuples, a stock count, a set of regulars, and sentiment scores on the reviews. And it dies from one keystroke. A customer asks for three muffins, the cashier types the word `three` at the quantity prompt, and `int()` cannot read it:

```
  Quantity: three
ValueError: invalid literal for int() with base 10: 'three'
```

One confused customer, and the register is dead: the stock, the regulars and the running revenue it held in memory go with it. The week has two halves. The first looks under the floorboards, at what Python's memory is actually doing, and meets a bug you cannot see by reading the code. The second makes the register survive what a real day throws at it: bad input, a missing file, zero guests. By the end, the revenue carries over from one day to the next.

Recap. Dictionaries and tuples, from week 4, and the sentiment analysis built on them. One question puts all three in seven lines. What does this print? Trace it one pass of the loop at a time, as page 6 does.

```
totals = {}
for item, qty in [("tea", 2), ("bun", 1), ("tea", 3)]:
    if item in totals:
        totals[item] = totals[item] + qty
    else:
        totals[item] = qty
print(totals)
```

pass	item, qty	item in totals?	totals after the pass
1	tea, 2	no: a new key	{'tea': 2}
2	bun, 1	no: a new key	{'tea': 2, 'bun': 1}
3	tea, 3	yes: add to it	{'tea': 5, 'bun': 1}

It prints `{'tea': 5, 'bun': 1}`. Each pass hands out one tuple, and `item, qty` unpacks its two halves. The first time a key appears, the `else` creates it; after that, the `if` adds onto it, so a key is never doubled. Two wrong answers tempt here: `{'tea': 3, 'bun': 1}`, as if the third pass replaced the 2 instead of adding to it, and a dictionary holding `'tea'` twice, which no dictionary can do. The keys print in the order they arrived, `tea` first. And this loop is the engine of week 4's sentiment report: `review_scores` adds each review's score into `totals[product]` the same way, counts the reviews beside it, and divides one by the other at the end.

When you forget which brackets build what, ask Python. `type()` names what a value is, and a type called with nothing in its parentheses shows its empty form:

```
>>> print(type({}))
<class 'dict'>
>>> print(list(), dict(), tuple())
[] {} ()
```

Curly brackets alone make a dictionary, so an empty set has no brackets of its own: it is `set()`, as the café's `regulars = set()` writes it.

Week 4's follow-up is on the Q&A page, under Week 4.

1 · Memory

Slides, pages 7 to 13.

Since week 1, every variable has been a name pointing at an object. This section makes the pointing visible, then shows the bug it causes: a bug that crashes nothing and quietly changes data you thought was safe.

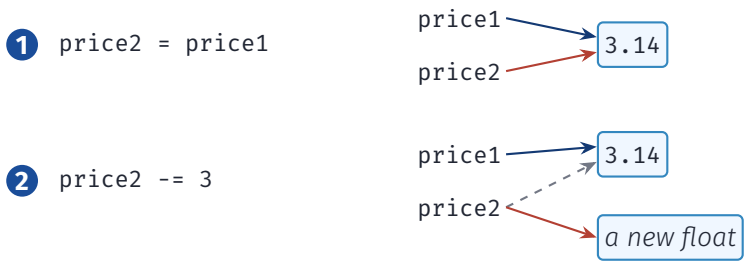
`id()`: where variables really live

A variable points to an object, as it has since week 1, and `id()` lets you watch it happen: it returns an object's identity, a number unique to that object while it exists, like a house address. Two names with the same identity are two labels on one object.

```
>>> price1 = 3.14
>>> price2 = price1
>>> print(id(price1) == id(price2))
True
>>> price2 -= 3
>>> print(id(price1) == id(price2))
False
>>> print(price1)
3.14
```

Print the two identities themselves and you get two long, equal numbers. They differ from machine to machine and from one run to the next, so only the sameness matters.

❶ `price2 = price1` copies the arrow, never the value: one float, two names, one identity. ❷ `price2 -= 3`, week 1's shorthand for `price2 = price2 - 3`, builds a new float and moves `price2` onto it, so the identities differ, and `price1` keeps pointing at 3.14.



This is week 1's rebinding rule, finally visible, and it explains week 4's in-place mutation lesson from the inside. In `for n in numbers: n = n * 2`, the loop points the name `n` at each element in turn, and `n = n * 2` points `n` at a new number. No seat of the list is touched, which is why that loop doubled nothing. On numbers and strings, rebinding is harmless: neither can change, so sharing one is safe.

Two cautions keep `id()` honest. An identity is unique only while its object exists: once nothing points at an object, Python throws it away and may hand its number to the next object it makes, so two values that never existed at the same moment can show the same number. And equal values typed separately may or may not share one object, and which ones do changes from one Python to the next. In the shell, two names given `5` share one:

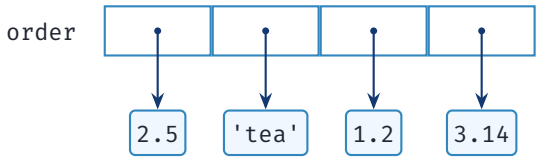
```
>>> small1 = 5
>>> small2 = 5
>>> print(id(small1) == id(small2))
True
```

Two names given `1000` get one each up to Python 3.14, and share one from Python 3.15. That is Python's own housekeeping, and harmless for the same reason: a number cannot change, so sharing one costs nothing. Never build on it. Compare values with `==`, and keep `id()` for watching what a name points at. A list is different, because a list can change in place.

Lists hold references too

Page 8. Week 4's row of seats, up close: a seat does not hold a value, it holds an arrow to one, an object that lives somewhere else in memory. Take a row of four values:

```
>>> order = [2.50, "tea", 1.20, 3.14]
>>> print(order)
[2.5, 'tea', 1.2, 3.14]
```



The moves are week 4's. `order[3] *= .99` builds a new number and re-points seat 3 at it; the old number is cleaned up if nothing else points at it, week 1's garbage collection. `del order[1]` takes the arrow out of the row, and the object it pointed at goes only when nothing references it anymore. The arrows are what is new, and they start to matter the moment two names point at the same list.

The aliasing bug

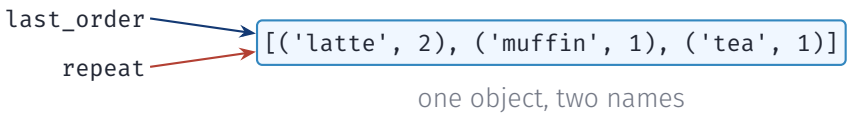
Page 9. A feature every café wants: repeat the last order, for the customer who wants the same again.

Poll. After `a = [1]`, then `b = a`, then `b.append(2)`, what does `print(a)` show: `[1, 2]`, `[1]`, `[2]`, or an error? It shows `[1, 2]`. `b = a` made a second name for the list, not a second list, so appending through `b` shows through `a`.

The café version does exactly the same with real data. Type it into the shell:

```
>>> last_order = [("latte", 2), ("muffin", 1)]
>>> repeat = last_order              # "copy"... or is it?
>>> repeat.append(("tea", 1))        # the new customer adds a tea
>>> print(last_order)
[('latte', 2), ('muffin', 1), ('tea', 1)]
```

The new customer's tea landed in the previous customer's order. `repeat = last_order` made a second name, not a second list. Both names point at the same object, so a change through either name shows through both:



one object, two names

No crash, no error message, just wrong data on a receipt. That is what makes aliasing the most invisible bug you will meet this term: nothing warns you, and the code reads as if it were right.

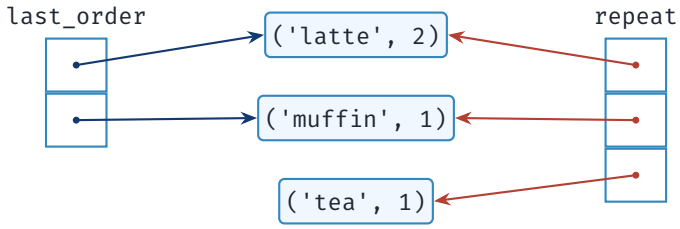
Real copies, and how deep they go

Page 10. Three idioms build a new list: the `list()` constructor, the clearest of the three; a full slice, `last_order[:]`, which copies from the first seat to the last; and a comprehension, `[x for x in last_order]`, week 4's tool. Each gives the copy its own identity, so appending to one leaves the other alone:

```
>>> last_order = [("latte", 2), ("muffin", 1)]
>>> repeat = list(last_order)
>>> repeat.append(("tea", 1))
>>> print(last_order)
[('latte', 2), ('muffin', 1)]
>>> print(id(repeat) == id(last_order))
False
>>> print(id(last_order[0]) == id(repeat[0]))
True
```

Look at the last line. The list is new, but the things it points at are the same objects: a new row of arrows, aimed at the same values. That is a **shallow copy**. With numbers, strings or tuples inside, it is all you need, since none of them can change. With lists inside, it is a trap.

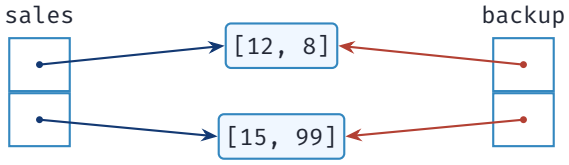
Drawn, the copy has seats of its own, on the right. Its first two arrows aim at the very tuples `last_order`'s seats aim at, and only its third seat, the appended tea, is its own. Appending added a seat to `repeat` alone, and the shared tuples are safe because a tuple cannot change.



Page 11. Take the corner of week 4's sales grid, a list of rows:

```
>>> sales = [[12, 8], [15, 11]]
>>> backup = list(sales)                # "backed up"... shallowly
>>> sales[1][1] = 99
>>> print(backup[1][1])
99
```

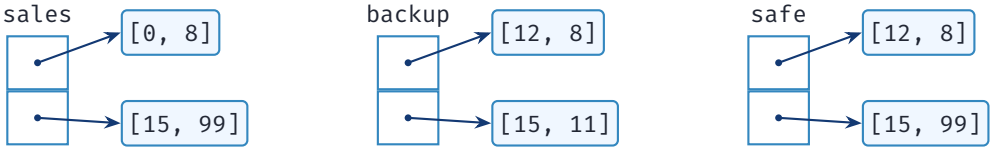
The outer list was copied, and the rows inside it are shared, so a change to a row shows through both lists:



Two real fixes. Copy every row yourself, a slice inside a comprehension, or import the `copy` module and let `deepcopy` copy everything, all the way down:

```
>>> sales = [[12, 8], [15, 11]]
>>> backup = [row[:] for row in sales]   # copy every row yourself
>>> sales[1][1] = 99
>>> print(backup[1][1])
11
>>> import copy
>>> safe = copy.deepcopy(sales)          # or: copy EVERYTHING, recursively
>>> sales[0][0] = 0
>>> print(safe)
[[12, 8], [15, 99]]
>>> print(sales)
[[0, 8], [15, 99]]
>>> print(backup)
[[12, 8], [15, 11]]
```

Each name now owns its rows, so a change through one reaches no other. `backup` kept its 11 when `sales[1][1]` became 99; `safe`, made after that change, kept its 12 when `sales[0][0]` became 0:



`deepcopy` is the sledgehammer: always correct, and hungry for memory and time on a large structure. Know the shape of your data and copy exactly as deep as you need. Immutable values, numbers, strings and tuples, never need copying at all: they cannot change, only be replaced.

Good practice. Decide who owns each list. When two parts of a program hold the same list, write down, in a docstring or a comment, which part may change it, and make the copy at the point where ownership passes. Most aliasing bugs are two owners who each believed they held the only copy.

In the project. The interview asks your group to justify its design and programming choices, part B of the grading rubric, and a copy is one of them: know why each copy in your code is there, and how deep it goes.

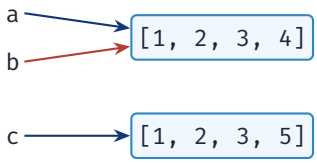
Follow the pointers

Page 12.

Poll. Predict all four prints before you run anything, then draw the boxes and arrows:

```
a = [1, 2, 3]
b = a
c = list(a)
b.append(4)
c.append(5)
print(a)
print(b)
print(c)
print(id(a) == id(b), id(a) == id(c))
```

`a` and `b` both print `[1, 2, 3, 4]`: one list under two names, and `b.append(4)` went into it. `c` prints `[1, 2, 3, 5]`: its own list, a real copy made before either append. The last line prints `True False`. Drawn, it is this:



If you got all four, references and copies hold no surprises for you.

Your turn: the stock template

Page 13. The café keeps a `standard_stock` template and starts each day from it:

```
>>> standard_stock = {"espresso": 50, "latte": 50, "muffin": 15}
>>> todays_stock = standard_stock        # the bug is on this line
>>> todays_stock["muffin"] = 0           # sold out by 9am
```

Poll. What has just happened to `standard_stock`? Check with a print:

```
>>> print(standard_stock["muffin"])
0
```

Its muffins went to zero too. `todays_stock = standard_stock` made a second name, not a copy, so the template and the day are one dictionary. The fix has the same shape as for lists: `dict(standard_stock)` makes a real copy.

And why does `template = "closed"` followed by `sign = template` never have this problem? Because a string cannot change. Pointing `sign` at a new string later rebinds `sign`; nothing reaches into the original. Mutable things alias dangerously, and immutable things are safe to share.

Common mistakes. Taking `=` for a copy, as above: the commonest of the three. Taking `list(...)` or `[:]` for a deep copy when the elements are lists themselves: the inner lists are still shared, and the shallow trap of page 11 follows. And naming a file of your own `copy.py`: in that folder, `import copy` finds your file before Python's module, and `copy.deepcopy` is not there.

Check yourself.

- **1.1** After these four lines, what are `menu["tea"]` and `spare["tea"]`?

```
menu = {"tea": 2.20}
today = menu
spare = dict(menu)
today["tea"] = 2.50
```

- **1.2** With `queue = ["ada", "bob"]`, which of `a = queue`, `b = queue[:]` and `c = list(queue)` still holds two entries after `queue.append("cy")`?
- **1.3** After these two lines, what do `rota == plan` and `id(rota) == id(plan)` give, and why?

```
rota = ["mon", "tue"]
plan = rota[:]
```

See also. Sheet 5, exercise 1. The Function reference, Names and copies. Automate the Boring Stuff chapter 6: its part on references now reads as revision.

2 · Program structure

Slides, pages 14 to 17.

A program is easier to read, to test and to reuse when every file is laid out the same way. This section gives that layout, the safe way to import, two shapes to avoid, and the meaning of the two lines that end every café file.

The canonical layout

Every serious Python file has the same skeleton, and the café's files have had it since week 3:

```
# 1. Import statements
import register_sayings

# 2. Global constants
DISCOUNT_RATE = 0.10

# 3. Function declarations
def apply_discount(total): ...

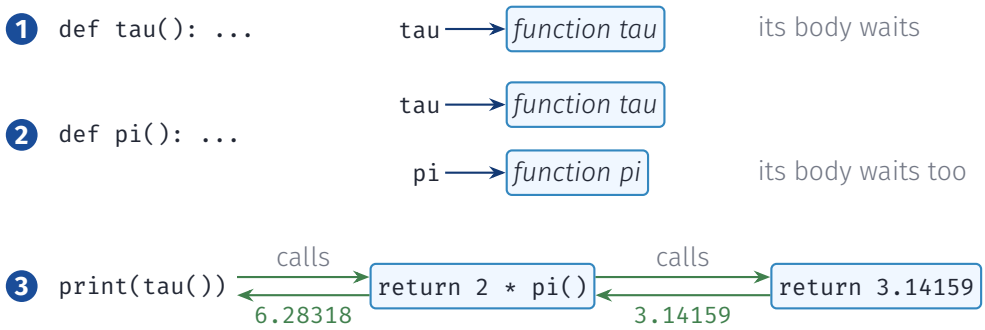
# 4. Main code: the only line that RUNS anything
if __name__ == "__main__":
    main()
```

Imports first, so you always know what is loaded, and because importing a file runs it, which is week 2's rule. Constants next, so every function below can see them. Definitions before use, because Python reads downward and runs a function's body only when the function is called. So a function may call another defined lower in the file, as long as both exist by the time the call runs:

```
>>> def tau():
...     return 2 * pi()
...
>>> def pi():
...     return 3.14159
...
>>> print(tau())
6.28318
```

Call `tau()` before `pi` is defined and Python stops with `NameError: name 'pi' is not defined`. Define everything first, run last.

Python reads the file from the top. ❶ `def tau():` makes the function and points the name `tau` at it, and the body waits. ❷ The same for `pi`. ❸ Only the last line calls anything: it calls `tau`, whose body now calls `pi`, which exists by then, and the values come back, 3.14159 and then 6.28318. A body runs when its function is called, never when it is defined.



Import styles: one is dangerous

Page 15. One line of the skeleton hides a real choice:

```
from register_sayings import *        # dumps every name into your file
from register_sayings import welcome  # explicit, but where is it from?
import register_sayings               # register_sayings.welcome(): traceable
```

The star import is the dangerous one. It pours every name from the module into your file, and a function you define later with the same name replaces the imported one without a word. From your café folder, where `register_sayings.py` lives:

```
>>> from register_sayings import *
>>> print(welcome("Ada"))
Welcome to The Daily Grind, Ada!
>>> def welcome(name):
...     return "Next!"
...
>>> print(welcome("Ada"))
Next!
```

(1) The star import copies the module's names into your file, so your `welcome` and `goodbye` point at the module's objects. (2) Your own `def welcome` points `welcome` at a new function, and nothing tells you the imported one is out of reach. (3) Plain `import register_sayings` puts one name in your file, the module itself, and every use reaches in through it, so nothing of yours can replace `register_sayings.welcome` by accident.



Plain `import register_sayings`, then `register_sayings.welcome(...)`, is the form to write: every use says where it comes from. That is week 2's advice, now with its reason. Two modules can both define a `welcome`, and only the module's name in front of each call keeps them apart. One more trap: circular imports. If `x.py` imports `y.py` and `y.py` imports `x.py`, one of them meets the other half-loaded. If you feel the urge, the structure of your program is asking for a redesign.

Spaghetti, and the function that eats itself

Page 16. A function should do its job and return. Two shapes feel clever and age badly. The first is a function that calls itself to repeat its work:

```
def square_loop():
    nbr = int(input("Number (0 quits): "))
    if nbr == 0:
        return
    print("Square:", nbr ** 2)
    square_loop()        # <- recursion, standing in for a loop
```

Each call waits for the one it started, so every number typed stacks up one more unfinished call. Python allows about a thousand, then stops with `RecursionError: maximum recursion depth exceeded`. The same job as a loop never runs out:

```
def square_loop():
    nbr = int(input("Number (0 quits): "))
    while nbr != 0:
        print("Square:", nbr ** 2)
        nbr = int(input("Number (0 quits): "))
```

Recursion has real uses, in later courses; standing in for a loop is not one of them. The second shape is functions that call each other back and forth to keep a program going, each calling the next instead of returning: spaghetti. Loops loop, and functions return.

`main()`, `__name__`: the café's last two lines, explained

Page 17. Since week 3, every café file ends with two lines we never explained:

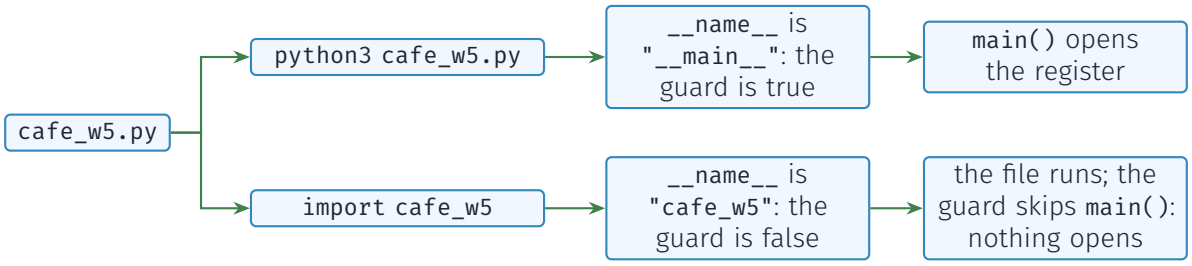
```
if __name__ == "__main__":
    main()
```

Every file has a name, kept in the variable `__name__`. Run a file directly and its name is the string `"__main__"`; import it and its name is the module's own, `"cafe_w5"`. So the guard reads: run `main()` only when I am the program, not when I am a library. `register_sayings.py` needs no guard: it only defines things, week 2's rule for a module, so importing it prints nothing anyway. From your café folder, try it:

```
>>> import cafe_w5
>>> print(cafe_w5.__name__)
cafe_w5
>>> print(__name__)
__main__
>>> print(cafe_w5.apply_discount(20.00))
18.0
```

The import printed nothing and opened no register, because `main()` sits under the guard, and yet every function came with it. Another program can now reuse `apply_discount` without a register ever opening: every café file is both a program and a library, and section 5 cashes this in.

The same file takes two roads. Run it, and its `__name__` is `"__main__"`: the guard is true, and `main()` opens the register. Import it, and its `__name__` is `"cafe_w5"`: the file still runs from the top, its import, its constants and its definitions, but the guard is false, so `main()` never starts, and every function is ready to call through the module's name.



Watch both names happen. Add one line at the very end of `register_sayings.py`:

```
print("My name is", __name__)
```

Run the module itself, `python3 register_sayings.py`, and it prints `My name is __main__`. Start the café, `python3 cafe_w5.py`, and `My name is register_sayings` prints before the cashier prompt: the café's first line imports the module, and importing a file runs it. Then take the line out again, since a module only defines things.

One rule comes with it: a file you want to import must be named like a variable, by week 1's rule: letters, digits and underscores, never a digit first, plus `.py`. `register_sayings.py` imports; a file called `my register!.py` cannot be named in an `import` line. It still runs as a program, since `python3` accepts any file name; only `import` needs a name it can read as a variable.

> **In the project.** Page 17's "your project will lean on exactly this" means your `agent.py`: the agent contract, project document 03, says how it is loaded and what it must hold.

Common mistakes. The guard spelled `if __name__ == "main":` compares the name with the wrong string, so `main()` never runs and the program ends without a word. A guard left out runs `main()` on every import, so a test that imports the file starts by asking for a cashier's name. A file name with a space or a hyphen, such as `cafe-w5.py`, cannot be imported: the hyphen reads as a minus sign. And a function that returns, called on a line of its own in a file, prints nothing: `welcome("Ada")` builds the sentence and drops it. The shell shows the value of a call typed on a line of its own, and a file never does, so write `print(welcome("Ada"))`.

Check yourself.

- **2.1** Which of these files can another program import: `tips.py`, `tip calculator.py`, `2tips.py`, `tips_v2.py`?
- **2.2** `shop.py` ends with a bare `main()` on its last line, with no guard. What happens when `check_shop.py` runs `import shop`?
- **2.3** Put these four parts of a file in order: `RATE = 0.10`, a `def` block, the guard with `main()`, and `import sys`.

See also. Sheet 5, exercise 6, part (c). The Function reference, Program layout.

3 · The three kinds of error

Slides, pages 18 to 21.

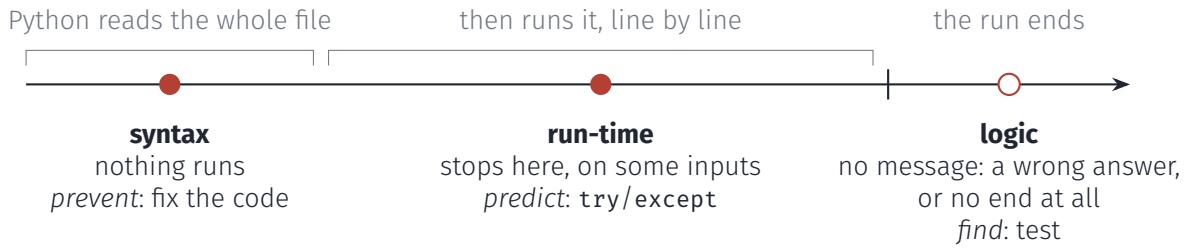
Poll. In one or two words: how has one of *your* programs died this term? Name the crash, not the feeling. Syntax errors came up most, a missing bracket among them, and loops that never stop came up too: those belong in the third box below, since nothing crashes. Whatever you named, it fits one of three boxes:

Kind	When it strikes	Your defense
1. Syntax	before the program even starts	**prevent:** fix the code
2. Run-time	mid-execution, on certain inputs	**predict:** `try`/`except`
3. Logic	never crashes, just *wrong*	**find:** test relentlessly

A syntax error stops Python before the first line runs: it cannot read the program. A run-time error happens while the program runs, on some inputs and not others, so you predict where it can strike and guard that spot. A logic error never crashes at all: the program runs to the end and gives the wrong answer, and only testing finds it.

A program's life has two phases: Python first reads the whole file, then runs it line by line. The syntax family strikes in the reading, so nothing runs at all. A run-time error strikes mid-run, on

some inputs and not others. A logic error never strikes: the run ends with a wrong answer, or never ends at all, like the loops in the poll.



Triage: four broken registers

Page 20. Three registers that cannot even start, each with the message Python prints:

```
if open_sign = True
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?

if open_sign == True
SyntaxError: expected ':'

if open_sign == True:
    print("We are open")
   print("Come in!")
IndentationError: unindent does not match any outer indentation level
```

The single `=` assigns, and a condition needs `==`, week 2's old enemy. The second line has the right comparison and no colon. The third indents its second `print` by three spaces where the first used four, so the line matches no level above it. All three are the syntax family: caught before anything runs, so you prevent them.

The run-time errors bite once the program runs, all of them familiar by now:

```
>>> total = 212
>>> total / 0
ZeroDivisionError: division by zero
>>> print(unknown)
NameError: name 'unknown' is not defined
>>> "3" + 0.14
TypeError: can only concatenate str (not "float") to str
```

And the fourth register runs perfectly and prints a clean receipt with the discount applied twice. From your café folder:

```
>>> import cafe_w5
>>> print(cafe_w5.apply_discount(cafe_w5.apply_discount(20.00)))
16.2
```

A 20.00 order should cost 18.00. No error anywhere, 1.80 short on every such order: a logic error, the expensive kind.

A logic error can hide in one character: `%` is not percent. It gives the remainder of a division, and ten percent of fifty takes a multiplication:

```
>>> print(50 % 10)
0
>>> print(50 * 0.10)
5.0
```

A program that reads `%` as percent runs to the end without a word, and every number it computes that way is wrong.

What sloppy error handling costs

Page 21. Three stories put a price on it.

- **USS Yorktown, 1997.** A crew member typed a zero into a database field, and the division by zero cascaded through the ship's network until its propulsion failed. A warship, dead in the water, stopped by a `ZeroDivisionError`.
- **Mariner I, 1962.** The probe was destroyed 293 seconds after launch. Its hand-written guidance equations put a bar over one symbol, meaning "use the smoothed value". The bar never reached the program, which steered on raw, jumpy readings. One missing symbol.
- **Mars Climate Orbiter, 1999.** A 125-million-dollar spacecraft was lost because one team's software worked in pound-seconds and another's in newton-seconds. No crash, no error message: a logic error, carried all the way to Mars.

The register guarding its quantity prompt is the same discipline, at café scale.

> **Good practice.** Let a tool catch the syntax family before Python does. An editor with a linter, such as VS Code's Python support or `ruff`, underlines a missing colon or a stray `=` as you type. The course sets up no linter, and one changes nothing you submit: it only finds the mistake sooner.

Common mistakes. Reading a traceback from the top. Python prints where the error came from first and what it is last, so start at the bottom: the last line names the kind of error and its message, and the lines just above it name the file, the line number and the code.

Check yourself.

- **3.1** Name the kind of each error, and its defense: (a) `def total(a, b)` with no colon; (b) `len(5)`; (c) an average of three prices computed as `sum(prices) / 2`.
- **3.2** You run this file, ready to type `abc` at the prompt. What do you see, and why can the `except` not help?

```
print("Split the bill")
try:
    guests = int(input("Guests: "))
except ValueError:
    print("Digits only.")
    guests = 1
if guests == 4
    print("A table for four")
```

See also. Sheet 5, exercise 2. The Function reference, Errors and exceptions.

4 · Exceptions

Slides, pages 22 to 27.

`try` and `except`: the anatomy

Page 22.

Poll. Four prints and one bomb. Which lines print? Trace it by hand first:

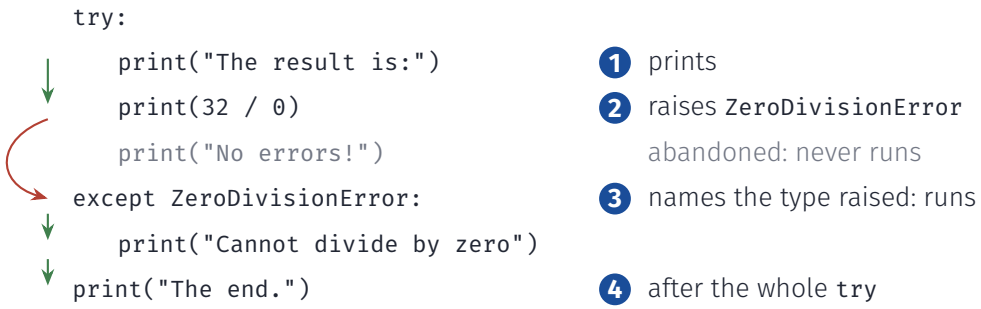
```
try:
    print("The result is:")
    print(32 / 0)
    print("No errors!")
except ZeroDivisionError:
    print("Cannot divide by zero")
print("The end.")
```

Three lines:

```
The result is:
Cannot divide by zero
The end.
```

The `try` block runs until something raises. At that instant the rest of the block is abandoned, so `No errors!` never prints. Python looks for an `except` that names the type just raised, runs it, then carries on after the whole `try`, and the program lives. Had no `except` matched, the exception would have escaped and crashed the program as usual.

Follow the run down the code. ❶ The first `print` runs. ❷ `32 / 0` raises, and the rest of the block is abandoned. ❸ Python finds the `except` that names `ZeroDivisionError` and runs its block. ❹ Then it carries on after the whole `try`.



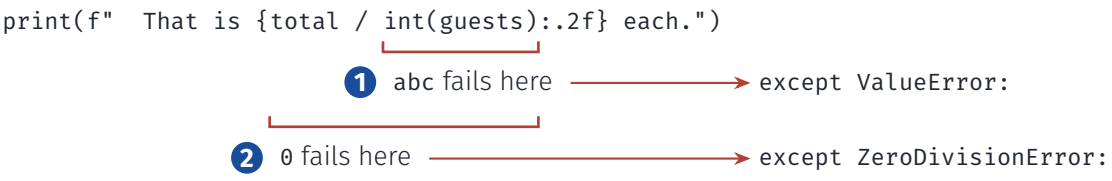
Two disasters, two catches: split the bill

Page 23. A friendly feature hides two different disasters, so it needs two catches:

```
def split_bill(total):
    """Offer to split the bill -- two different disasters, two different catches."""
    guests = input("  Split between how many? (ENTER to skip): ").strip()
    if guests == "":
        return
    try:
        print(f"  That is {total / int(guests):.2f} each.")
    except ZeroDivisionError:
        print("  You cannot split a bill between zero people...")
    except ValueError:
        print("  That does not look like a number of people.")
```

One `try` around the division, and two `except` clauses: the one whose type matches the raised error runs. Zero guests raise `ZeroDivisionError` in the division; letters raise `ValueError` in `int()`. When several failures deserve the same answer, one clause can name them all, in parentheses: `(FileNotFoundError, ValueError)`, as page 27 does.

Inside the one line, two things can fail, in order. ❶ `int(guests)` runs first, and letters fail there. ❷ Then the division runs, and a zero fails there. Each error goes to the `except` that names it.



To run it, start `python3 cafe_w5.py` from your café folder, type `Ada` as the cashier, choose `1`, press Enter to skip the customer's name, order `latte` and `1`, then `done`, and answer `0` at the split prompt. After the receipt:

```
  TOTAL                 3.20   (1 items)
  Split between how many? (ENTER to skip): 0
  You cannot split a bill between zero people...
```

Zero guests, and the register shrugs instead of dying. Answer `abc` instead, and the other clause answers:

```
  Split between how many? (ENTER to skip): abc
  That does not look like a number of people.
```

The lazy catch, and the honest ones

Page 24. Three versions of one file reader, each more honest than the last:

```
except:                          # 1) BARE: catches EVERYTHING -- even your typos.
    print("Error. Aborted.")     #    You'll never know what went wrong. Don't.

except OSError:                  # 2) NAMED: each cause gets its own message.
    print("Could not open the file.")
except ValueError:
    print("A line wasn't a number.")

except Exception as error:       # 3) INSPECTABLE: the exception is an OBJECT --
    print(f"{type(error)} in read_data()")   #    print its type while debugging.
```

The bare `except` catches everything, a misspelled variable's `NameError` included, and says `Error. Aborted.` for all of it. The named version gives each cause its own message; `OSError` is the family `FileNotFoundError` belongs to, so it covers any file that will not open. The third captures the exception as an object, `error`, so you can print its type while you debug. And why not wrap the whole program in one big `try`? Because then you lose where it broke, and you cannot recover from a failure you cannot locate. Catch small, and catch close to where the trouble starts.

Do you need to know an error's name before you can catch it? Yes, and the quickest way to learn it is to make it happen once. Type the failing line in the shell and read the last line Python prints:

```
>>> 4 / 0
ZeroDivisionError: division by zero
```

So `except ZeroDivisionError:` is the clause to write. Inside a program, the third version above finds the name for you. When `int()` meets the word `abc` inside that `try`, it prints the line below, and you know which `except` to write:

```
<class 'ValueError'> in read_data()
```

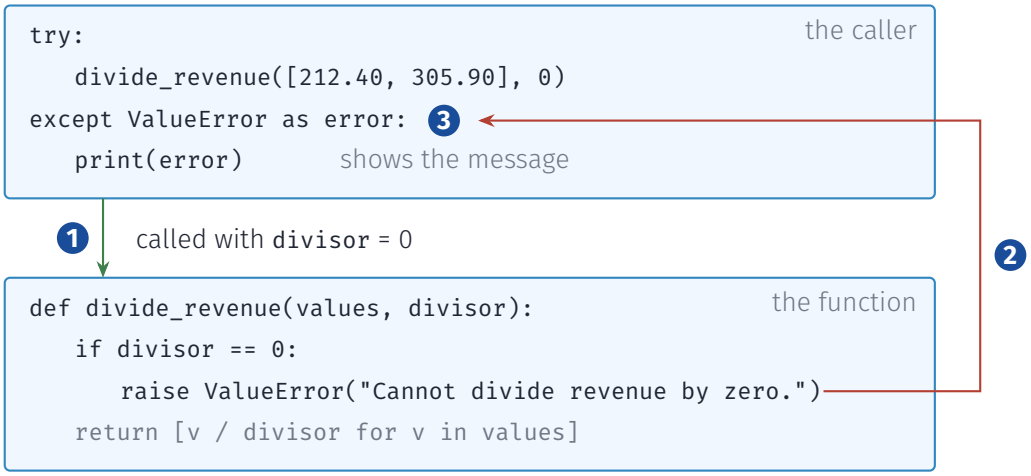
> **Good practice.** Keep a `try` block to the line that can fail. Ten lines inside a `try` catch a `ValueError` from any of them, including the one you did not expect; a `try` around the single `int(...)` call says exactly what you are guarding against.

`raise`: refusing loudly

Page 25. You can throw exceptions as well as catch them. A function states its rule and refuses when the rule is broken; the caller decides what the refusal means:

```
>>> def divide_revenue(values, divisor):
...     if divisor == 0:
...         raise ValueError("Cannot divide revenue by zero.")
...     return [v / divisor for v in values]
...
>>> print(divide_revenue([212.40, 305.90], 2))
[106.2, 152.95]
>>> try:
...     divide_revenue([212.40, 305.90], 0)
... except ValueError as error:
...     print(error)
Cannot divide revenue by zero.
```

1 The caller calls `divide_revenue` with a divisor of 0. 2 The function checks its rule and raises: the `ValueError`, carrying its message, leaves the function at once, and the `return` line never runs. 3 It arrives in the caller, where `except ValueError as error:` catches it and names it `error`, and `print(error)` shows the message.



Refusing loudly is one road. Replying is the other: the function hands back a pair, `(accepted, message)`, and the caller reads it. Week 7 builds the register's heart on the reply. Both roads are in this week's files, side by side on an oversell: `w05_oversell_raise.py` refuses with an exception, `w05_oversell_reply.py` with a reply. Run both and compare what the caller has to write.

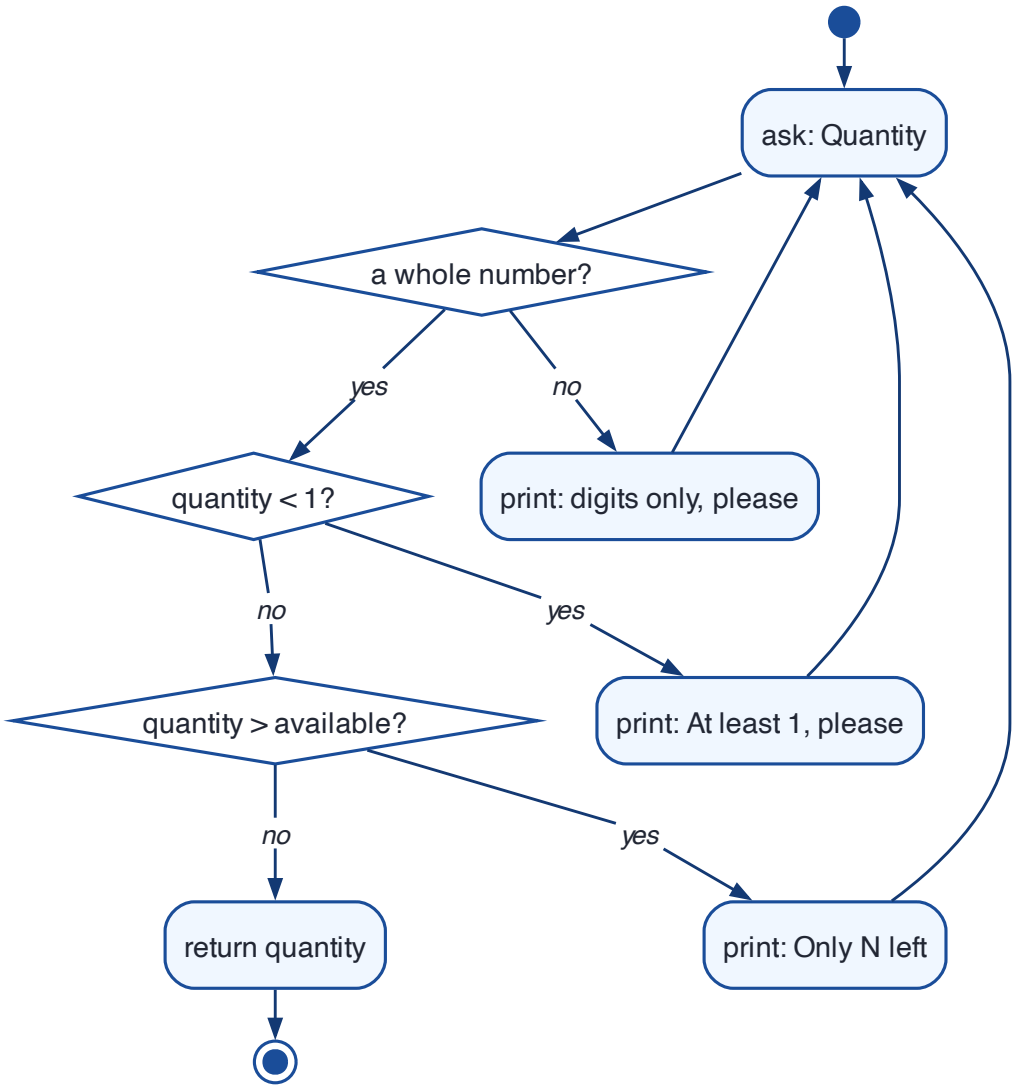
The unkillable prompt

Page 26. Week 3's validation loop plus this week's `except` make a prompt nothing can kill:

```
def ask_quantity(product, available):
    """The unkillable prompt: keep asking until we get a valid, in-stock quantity."""
    while True:
        try:
            quantity = int(input("  Quantity: "))
        except ValueError:
            print("    That doesn't look like a number -- digits only, please.")
            continue                    # back to the top -- ask again
        if quantity < 1:
            print("    At least 1, please.")
        elif quantity > available:
            print(f"    Only {available} {product}(s) left.")
        else:
            return quantity
```

It loops forever. If `int()` chokes, the `except` apologizes and `continue` jumps straight back to the top of the loop to ask again. If the number is under one, or more than the stock, it says so and loops. Only a valid, in-stock number reaches the `return`.

Drawn, the prompt has four ways out of its question. Letters get their message, and `continue` sends them straight back to it; a number under one or over the stock gets its message and goes round the loop; only a valid number leaves, through the `return`.



Run the café, type `Ada` as the cashier, choose `1`, skip the name, order `muffin`, and try your worst:

```
Product (or 'done'): muffin
Quantity: three
  That doesn't look like a number -- digits only, please.
Quantity: 999
  Only 15 muffin(s) left.
Quantity: 2
Product (or 'done'): done
======== The Daily Grind =========
muffin       x  2     4.80
==================================
TOTAL                 4.80   (2 items)
```

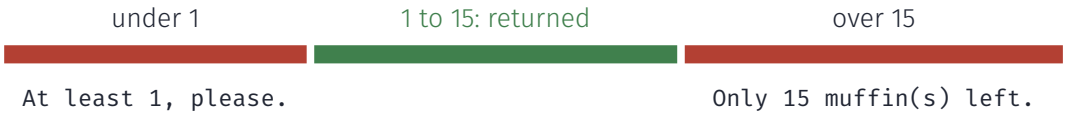
The word `three`, caught and asked again; 999, refused; 2, accepted. One new word rode along: `continue` skips straight to the loop's next pass. Week 4's `break` leaves a loop; `continue` goes around again.

The prompt has one case it cannot survive. Sell all 15 muffins in one order, then order a muffin again: `take_order` calls `ask_quantity("muffin", 0)`, and every answer is refused.

```
Product (or 'done'): muffin
Quantity: 1
  Only 0 muffin(s) left.
Quantity: 0
  At least 1, please.
Quantity: 2
  Only 0 muffin(s) left.
Quantity:
```

(1) With 15 muffins on the shelf, the numbers from 1 to 15 reach the `return`; a number under 1, or over the stock, is refused and asked again. (2) With 0 left, "over the stock" starts at 1, exactly where "at least 1" ends. The window between the two refusals is empty, so no number leaves the loop, and letters only get their own message.

(1) 15 muffins left



(2) 0 muffins left: no number is returned



Only stopping the program ends it: Ctrl-C stops the whole register, and the running revenue, written to `revenue.txt` only at closing, goes with it. Nothing crashed and no `except` fired, so this is the third kind of error, found only by using the program the way a busy day does. The fix refuses before it asks: in `take_order`, a product with nothing on the shelf is turned away before `ask_quantity` is called. Two lines go right after the branch that refuses a product the café does not sell, before the `else`:

```
        elif stock[product] == 0:                # sold out: asking would loop forever
            print(f"    Sorry, no {product}(s) left.")
```

The same order is now refused at once, and the register asks for the next product:

```
Product (or 'done'): muffin
  Sorry, no muffin(s) left.
Product (or 'done'):
```

The café files carry these two lines from week 6 on. Add them to your own `cafe_w5.py`.

The first-morning fix: the money lasts overnight

Page 27. Week 3 previewed the crash: on the café's very first morning there is no `revenue.txt`, and opening it raises `FileNotFoundError`.

Poll. Type the header that catches a missing file: after `try:` and its block, which line comes next? This one:

```
except FileNotFoundError:
```

The café catches a little more, a missing file and a corrupted number, in one grouped clause:

```
def load_revenue():
    """Read the running revenue, or start at 0.0 -- the Week-3 crash, handled."""
    try:
        f = open("revenue.txt")
        amount = float(f.readline())
        f.close()
        return amount
    except (FileNotFoundError, ValueError):
        return 0.0
```

First morning ever: no file, so `0.0`, no drama. A corrupted file: `float()` raises `ValueError`, so `0.0` again. Any other morning: yesterday's money is in the drawer. Try both branches from your café folder. Delete `revenue.txt` if a run of yours left one, start the café, type `Ada`, choose `1`, skip the name, order `latte` and `2`, then `done`, skip the split, and close with `0`:

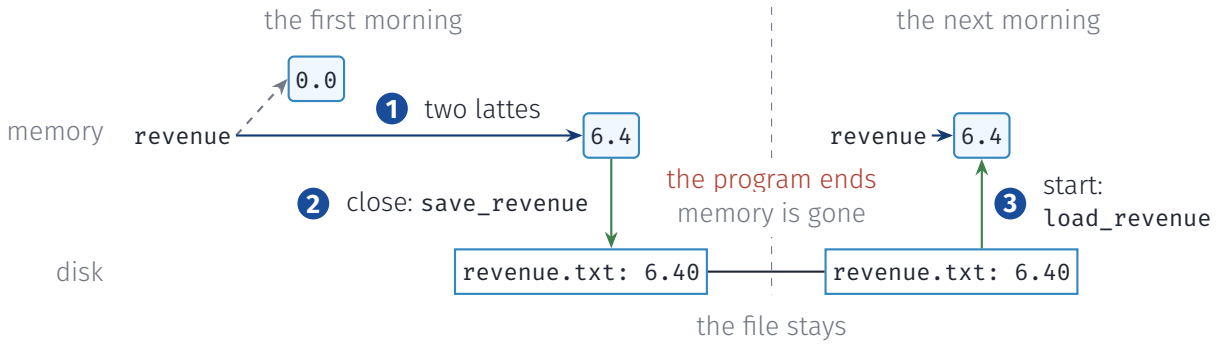
```
Carried over from yesterday: 0.00
...
Closed. Revenue on file: 6.40
```

Start it again and type `Ada`: after the welcome, it prints `Carried over from yesterday: 6.40`. Close with `0`. Close the register, close the terminal, open the next morning, and the money is still there: the café finally has a past.

From the shell in your café folder, the two functions show the same handover:

```
>>> import cafe_w5
>>> cafe_w5.save_revenue(6.4)          # closing time, after the two lattes
>>> print(cafe_w5.load_revenue())     # the next morning
6.4
```

Two lifetimes are at work. ❶ The running revenue lives in memory: two lattes move `revenue` from 0.0 to 6.4, and memory ends with the program. ❷ At closing, `save_revenue` writes it to `revenue.txt` as `6.40`, and the file outlives the program. ❸ The next morning `load_revenue` reads it back into memory.



> **Good practice.** Close what you open, on every path. In `load_revenue`, a corrupted file raises inside `float(...)`, before `f.close()` runs, so the file stays open until Python cleans up after the function. `with open("revenue.txt") as f:` closes the file however the block ends. The course writes `open` and `close` by hand and does not teach `with`; it is worth knowing for the Python you write after it. And a corrupted file is not a first morning: say so with a `print` before starting at zero, or yesterday's money vanishes without a word.

Common mistakes. Grouping types without parentheses. Python 3.13 and earlier refuse `except ValueError, TypeError:` with `SyntaxError: multiple exception types must`

`be parenthesized`; 3.14 accepts it, but not with `as error` after it. The parentheses work in every version, so write them: `(ValueError, TypeError)`. Catching the wrong type: an `except ValueError` around a division does not catch `ZeroDivisionError`, so the program still dies. An `except` with nothing under it: Python refuses the file before running any of it, `IndentationError: expected an indented block after 'except' statement on line 3` for an `except` on line 3. A clause with nothing to do says so with `pass`, the statement that does nothing. And a bare `except:` around a typo, which turns a misspelled name into `Error. Aborted.` and hides the line you need to fix.

Check yourself.

- **4.1** What does `split_bill(3.20)` print when the guests are typed as `-2`, and which kind of error is that?
- **4.2** When `revenue.txt` holds the words `six forty`, which line of `load_revenue` raises, and which clause catches it?
- **4.3** What prints, and what happens next?

```
try:
    print("start")
    print(5 / 0)
except ValueError:
    print("caught")
print("end")
```

See also. Sheet 5, exercises 3, 4 and 5. The Function reference, Errors and exceptions. py4e chapter 3, its section on `try` and `except`: exercise 2 of that chapter, parked in week 3 because it needs `try`, is yours now, and so is chapter 5's exercise 1. Automate the Boring Stuff chapter 5, which raises, catches and asserts.

5 · Assertions and the first unit test

Slides, pages 28 to 33.

Assertions: tripwires for the programmer

Page 28. An `assert` points the other way: at the programmer.

```
>>> def repeat(char, times):
...     assert isinstance(char, str)
...     assert isinstance(times, int)
...     return char * times
...
>>> print(repeat("-=", 3))
-=-=-=
>>> repeat("-=", "20")
AssertionError
```

`isinstance(VALUE, TYPE)` answers `True` when the value is of that type, so the two asserts check that `char` is a string and `times` a whole number. Called correctly, the asserts are silent. Called with the string `"20"` instead of the number 20, the second one stops the program with an `AssertionError`, and the traceback gives the line, straight to the fault. `assert` is a statement, not a function, so there are no parentheses around the condition. If the condition is false, it stops the program loudly, here, now. The crash is the point: an assertion says "if this is ever false, my program is already wrong; don't limp on".

Audience matters. Assertions talk to the programmer. Bad input from a user gets `try` and `except`; an impossible state inside your own program gets an `assert`.

Four defenses, one decision

Page 29. For "this value must be right", you now own four tools:

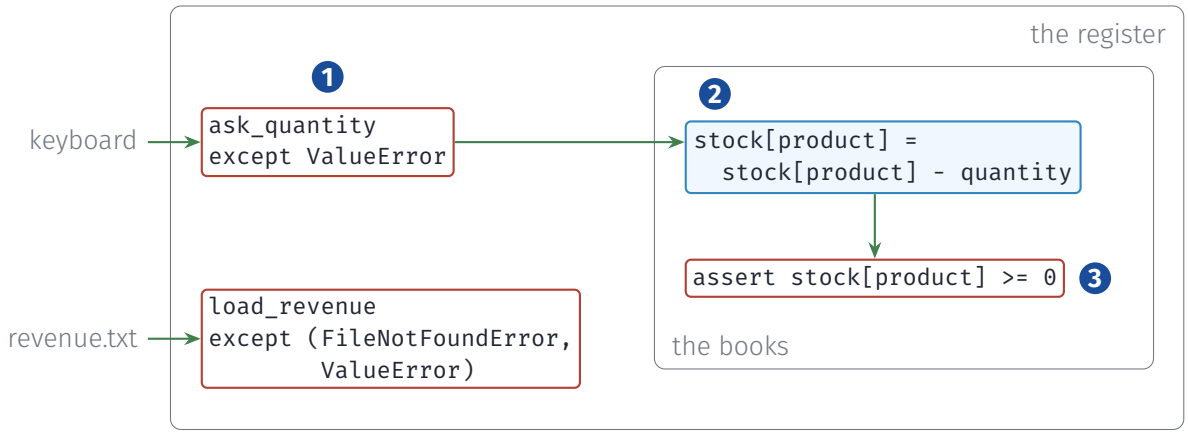
Tool	What it does with a violation	Use when
`if` + ignore	silently returns `None`, a logic error later	almost never
`try`/`except`	catches at the *call site*	expected, recoverable trouble: input, files
`raise`	refuses loudly, the caller decides	a function enforcing its contract
`assert`	crashes at the *fault site*	states that must never happen

The café makes two of the picks: `except` at the doors, where input and files come in, and `assert` on the books. Right after a sale takes its quantity off the stock:

```
stock[product] = stock[product] - quantity
assert stock[product] >= 0, "a sale broke the stock invariant"
```

The text after the comma is the message the `AssertionError` prints, so the crash explains itself. If that ever fires, `ask_quantity` has let something impossible through, and you want to know at once, not three receipts later. And the order is a rule worth keeping: **validate before you mutate.** `ask_quantity` checks first, the stock changes second, and the `assert` confirms third. Week 7 builds on exactly this rule.

Trouble from outside comes in through two doors, the keyboard and the files, and an `except` waits at each. Inside, the books carry an `assert` that should never fire. A sale goes through in the rule's order: ❶ `ask_quantity` validates, ❷ the stock changes, ❸ the `assert` confirms.



A file of asserts is a unit test

Page 30. In week 3 we tested the discount by hand: 9.99 unchanged, 10.00 unchanged, 20.00 down to 18.00. Honest work, and nobody repeats it after every change. So write the checks down. A file of asserts is a **unit test**: it pins the promises of one small unit, a function, and one command re-tests them all. Green means every promise still holds. Red stops the run at the first broken promise and names it, with its line number. The habit that makes the file worth keeping is one line long: run it after every change.

`check_discount.py`: the first unit test

Pages 31 and 32. The week-3 boundaries, written down:

```
import cafe_w5

assert cafe_w5.apply_discount(9.99) == 9.99      # under the threshold: untouched
assert cafe_w5.apply_discount(10.00) == 10.00    # ON the threshold: still untouched
assert round(cafe_w5.apply_discount(10.01), 2) == 9.01   # over: 10% off
assert cafe_w5.apply_discount(20.00) == 18.00    # comfortably over: 10% off

assert cafe_w5.receipt_line("latte", 3.20, 2) == "latte        x  2     6.40"
assert cafe_w5.receipt_line("espresso", 2.50, 10) == "espresso     x 10    25.00"

print("ALL CHECKS PASS (6 asserts).")
```

Under the threshold, untouched. On it, 10.00 exactly, still untouched, because week 3 chose greater-than, not greater-or-equal. Over it, ten percent off: 10.01 becomes 9.009, which the third line rounds to cents before comparing. And two receipt lines pin week 2's format specs to the exact character. Run it from your café folder:

```
$ python3 check_discount.py
ALL CHECKS PASS (6 asserts).
```

Look at what did not happen: no cashier prompt, no menu. Importing `cafe_w5` ran none of the register, because `main()` sits under the guard of section 2. An importable module is a testable module.

Now break a promise on purpose, the way a slip on the keyboard would. In a copy of your café folder, change `DISCOUNT_RATE = 0.10` to `0.1001`, still ten percent to the eye, and run the test again. The first three asserts pass, and the fourth stops the run. The traceback ends:

```
    assert cafe_w5.apply_discount(20.00) == 18.00    # comfortably over: 10% off
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError
```

The line above those names line 21 of the posted `check_discount.py`, which opens with comments the listing above leaves out. A 20.00 order less 10.01 percent is 17.998, not 18.0: a hundredth of a percent, and the red line names the promise it broke. Put the rate back to `0.10` and the file is green again.

Poll. Predict `apply_discount(25.00)`. It is `22.5`: ten percent off 25, and exact, so a bare assert holds. Add `assert cafe_w5.apply_discount(25.00) == 22.50`, change the message to seven asserts, run the file again, and it stays green.

> **In the project.** The brief ships a checker built on the same idea, `agent_unittest.py`, with the other starting files in the Project code and data folder. Run it after every change to your agent, the way you run `check_discount.py` after every change to the café.

> **Good practice.** When a file of asserts outgrows itself, a test runner takes over: Python's `unittest` module, or the `pytest` package, runs every test and reports every failure in one run, where a file of asserts stops at the first. Neither is on the project's library list, so in the project your tests stay plain asserts.

EAFP, duck typing, and a better assert

Page 33. Two mottos and one refinement. Do not `assert isinstance(...)` on every argument: that was a demonstration, not a style. Python's motto is that it is **easier to ask forgiveness than permission**: try the thing, and catch the failure if it comes. Its cousin is **duck typing**: if it walks like a duck and quacks like a duck, treat it as a duck. Care about what a value can do, not what it is.

A production-worthy assert guards an invariant, a truth that must hold, rather than a type:

```
>>> def normalize(values):
...     tot = sum(values)
...     result = [x / tot for x in values]
...     assert abs(sum(result) - 1.0) < 1e-9
...     return result
...
>>> print(normalize([1, 1, 2]))
[0.25, 0.25, 0.5]
```

`abs()` gives the distance from zero, so the assert asks whether the sum lands within a billionth of 1.0. Why the tolerance, rather than `== 1.0`? Week 1's float wall: a sum of floats can land near 1.0 rather than on it. Add a tenth ten times the way week 4's accumulators add:

```
>>> total = 0.0
>>> for share in [0.1] * 10:
...     total = total + share
...
>>> print(total)
0.9999999999999999
```

The tolerance also says how close is close enough. `10 / 3` prints as `3.3333333333333335`, so an assert that asks for exactly 3.33 fails, and one that asks only for 3.33 within a hundredth passes in silence:

```
>>> print(10 / 3)
3.3333333333333335
>>> assert 10 / 3 == 3.33
AssertionError
>>> assert abs(10 / 3 - 3.33) < 0.01
```

Invariants on floats always get a tolerance. One caveat: Python run with `-O`, optimized, strips every assert out. Never put logic your program needs inside one, only checks it should never fail.

> **Good practice.** Never put work inside an assert. If a function `record_sale(order)` saves a sale and returns `True`, then `assert record_sale(order)` saves nothing under `python -O`, because the whole line disappears; save first, then assert on what was saved.

Common mistakes. Parentheses around an assert's condition and its message:

```
assert (total >= 0, "negative")     # a tuple: never empty, so never false
```

That line asserts a tuple, which is never empty, so it can never fail, and Python says as much when it reads the file:

```
SyntaxWarning: assertion is always true, perhaps remove parentheses?
```

And comparing floats with `==` inside an assert, which fails a correct program the moment a result lands a hair off: round to what you care about, as `check_discount.py` does, or use a tolerance.

Check yourself.

- **5.1** Pick the tool for each: (a) one line of a price file reads `latte,n/a`, and the program must carry on with the other lines; (b) your own code calls `set_price(-3)`, which breaks the function's rule; (c) after a refund, a product's stock is higher than it was when the day began.
- **5.2** With `DISCOUNT_RATE` at `0.1001`, why does the line for 10.01 still pass, when the line for 20.00 fails?

See also. Sheet 5, exercises 5 and 6. The Function reference, Assertions and unit tests.

Where the café is now

Slides, pages 34 to 37.

`cafe_w5.py` keeps last week's menu, and almost everything stops breaking. The word `three` bounces off the quantity prompt, the bill splits safely with zero people included, the first morning starts at `0.0` instead of a crash, an `assert` guards the books, and the revenue carries over between days. One trap is left, the sold-out shelf of section 4, and two lines close it. The file itself narrates the canonical layout: imports, constants, functions, guarded main. One word in it is new, `pass`, the statement that does nothing. In `day_summary()`, `except FileNotFoundError: pass` means a missing receipts file gives a summary of zeros, not a crash. Keep `check_discount.py` beside the café, and run it after every change you make.

The itch left for next week is the owner's. She looks at the week's sales and asks: busiest hour? total revenue? average basket? biggest order? biggest day? Five of her questions, and each one a hand-written loop, or one import. Next week, on Tuesday: numpy, the day's numbers, vectorized.

What the exam can ask from this week, as page 35 lists it: classify errors into the three kinds and name the defense; predict aliasing outputs, the boxes-and-arrows trace; complete an `except` clause; state what `assert` does and who it talks to; read a short unit test and name the promise a red line breaks; spot why a bare `except:` is a bad idea.

Go through the checklist on page 37. Anything you cannot tick, read its section here again, then do the matching exercise on sheet 5. Self-check 5, open on Brightspace until Thursday 15 October at 23:59, asks the kinds of questions the exam asks. And the real homework is the one on page 36: go back through your own programs, make the copies real and the crashes caught.

Practice it closed-book

The exam is closed book, so practice the way it runs: from memory, on paper, then check.

- Cover the outputs in the blocks of sections 1 and 4, and the pictures beside them, and write each one exactly, quotes and brackets included; then look.
- Redraw page 12's boxes and arrows from memory, with the picture covered. Then make up a trace of your own with a dictionary, two names and one `dict(...)` copy, draw it, and run it to check.
- Write `ask_quantity` and `load_revenue` from memory, drop them into a copy of the café, and try to kill the register: letters, zero, a huge number, no `revenue.txt`, and, once the two sold-out lines are in `take_order`, a muffin after the fifteenth.
- List five errors you have met this term, with their messages. Classify each into the three kinds and name the defense, then check against page 19.
- Write a three-assert unit test for `receipt_line` from memory, run it, then break `receipt_line` on purpose and read the red line.

- In a few days, answer the check-yourself questions again without these notes, then take self-check 5 by hand, with Python closed.

Answers to check yourself

- **1.1** `2.5` and `2.2`: `today` is a second name for the same dictionary, so the change shows through `menu`; `spare` is a real copy, made before the change.
- **1.2** `b` and `c`: both are new lists. `a` is a second name for `queue`, so it holds three entries after the append.
- **1.3** `True` and `False`. `plan = rota[:]` is a real copy, so `plan` holds the same contents and `==` finds them equal, but it is a second object, with its own identity.
- **2.1** `tips.py` and `tips_v2.py`. A module's name must read like a variable: no space, and no digit at the start.
- **2.2** The import runs `shop.py` top to bottom, so `main()` runs during the import, and the shop starts before a single check does. Put `main()` under the guard.
- **2.3** `import sys`, then `RATE = 0.10`, then the `def` block, then the guard with `main()`.
- **3.1** (a) Syntax: Python stops with a `SyntaxError`, expected `':'`, before running anything; prevent it. (b) Run-time: `len(5)` raises a `TypeError`, because a number has no length; predict it. (c) Logic: no error, just a wrong number; find it with a test against a known value.
- **3.2** Not even `Split the bill`. Python reads the whole file before it runs any of it, and stops at line 7 with `SyntaxError: expected ':'`. No line runs, so the `try` never exists to catch anything. Prevent it: an `if` line ends with a colon.
- **4.1** `That is -1.60 each.`, after the two spaces every café line starts with. No exception is raised: a negative share is a logic error. A check that the number of guests is at least 1 would refuse it.
- **4.2** `amount = float(f.readline())` raises `ValueError`: the words `six forty` are no number `float()` can read. The grouped clause names `ValueError` beside `FileNotFoundError`, so it catches it, and the day starts at `0.0`.
- **4.3** `start` prints, then `5 / 0` raises `ZeroDivisionError`. The only `except` names `ValueError`, which does not match, so the error escapes and stops the program with a `ZeroDivisionError`; neither `caught` nor `end` prints.
- **5.1** (a) `try` and `except ValueError`, around the `float()` call inside the loop; (b) `raise`, inside `set_price`; (c) `assert`, right after the refund: a state that must never happen.
- **5.2** `apply_discount(10.01)` now gives 9.007999, and the line rounds it to cents before comparing: 9.01, as before. Rounding to cents hides a change that small. The line for 20.00 compares with no rounding, and 17.998 is not 18.0. The round is there for the correct rate too: 10.01 less ten percent is 9.009, three decimals, where a receipt shows cents.